

DeiMOS ADCS Simulator

Rigid Body Rotational Dynamics

Vivek Swamy
IIT Indore

21 June 2026

Contents

1	Introduction	2
2	Objective	2
3	Angular Momentum	2
4	Newton's Second Law for Rotation	3
5	Rotating Frame Derivative	3
6	Derivation of Euler's Rotational Equation	3
7	Angular Acceleration Formulation	4
8	Physical Interpretation	4
8.1	Applied Torque	4
8.2	Gyroscopic Coupling	5
9	Implementation in Python	5
10	Conclusion	6

1 Introduction

The rotational motion of a spacecraft is governed by the conservation of angular momentum and Newton's second law for rotational systems. This document derives Euler's rotational equations for a rigid spacecraft and establishes the mathematical foundation for the rotational dynamics module of the DeiMOS Attitude Dynamics and Control System (ADCS) simulator.

2 Objective

The objective of this document is to derive the equations governing the rotational motion of a rigid spacecraft and establish the mathematical foundation required to numerically simulate spacecraft attitude dynamics.

3 Angular Momentum

For a rigid body, the angular momentum vector is defined as

$$\mathbf{H} = \mathbf{J}\boldsymbol{\omega}$$

where

$$\mathbf{H} = \begin{bmatrix} H_x \\ H_y \\ H_z \end{bmatrix}$$

is the angular momentum vector,

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

is the angular velocity vector, and

$$\mathbf{J} = \begin{bmatrix} J_{xx} & J_{xy} & J_{xz} \\ J_{yx} & J_{yy} & J_{yz} \\ J_{zx} & J_{zy} & J_{zz} \end{bmatrix}$$

is the inertia tensor.

For a spacecraft whose body frame is aligned with the principal axes of inertia, the inertia tensor simplifies to

$$\mathbf{J} = \begin{bmatrix} J_x & 0 & 0 \\ 0 & J_y & 0 \\ 0 & 0 & J_z \end{bmatrix}.$$

This assumption is commonly used in spacecraft attitude dynamics because it simplifies the governing equations while preserving the essential physics.

4 Newton's Second Law for Rotation

Newton's second law for rotational motion states that the external torque acting on a rigid body equals the time rate of change of its angular momentum measured in an inertial frame.

$$\boldsymbol{\tau} = \left(\frac{d\mathbf{H}}{dt} \right)_I$$

where

$$\boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

is the external torque vector.

5 Rotating Frame Derivative

The simulator propagates the spacecraft state in the body frame rather than the inertial frame. Therefore, the relationship between inertial and body-frame derivatives must be used.

For any vector $\mathbf{A}$,

$$\left(\frac{d\mathbf{A}}{dt} \right)_I = \left(\frac{d\mathbf{A}}{dt} \right)_B + \boldsymbol{\omega} \times \mathbf{A}$$

Applying this relationship to the angular momentum vector yields

$$\left(\frac{d\mathbf{H}}{dt} \right)_I = \left(\frac{d\mathbf{H}}{dt} \right)_B + \boldsymbol{\omega} \times \mathbf{H}.$$

Substituting into Newton's second law gives

$$\boldsymbol{\tau} = \dot{\mathbf{H}} + \boldsymbol{\omega} \times \mathbf{H}$$

where

$$\dot{\mathbf{H}} = \left(\frac{d\mathbf{H}}{dt} \right)_B.$$

6 Derivation of Euler's Rotational Equation

Substituting

$$\mathbf{H} = \mathbf{J}\boldsymbol{\omega}$$

into the previous equation gives

$$\boldsymbol{\tau} = \frac{d}{dt}(\mathbf{J}\boldsymbol{\omega}) + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}).$$

For a rigid spacecraft, the inertia tensor remains constant in the body frame. Therefore,

$$\frac{d}{dt}(\mathbf{J}\boldsymbol{\omega}) = \mathbf{J}\dot{\boldsymbol{\omega}}.$$

Substituting,

$$\boldsymbol{\tau} = \mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}).$$

Rearranging yields Euler's rotational equation:

$$\boxed{\mathbf{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})}$$

This equation governs the rotational motion of the spacecraft.

7 Angular Acceleration Formulation

To obtain the angular acceleration explicitly, multiply both sides by the inverse inertia tensor.

$$\mathbf{J}^{-1}\mathbf{J}\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}[\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})]$$

which simplifies to

$$\boxed{\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}[\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})]}$$

This is the equation implemented in the rotational dynamics simulator.

8 Physical Interpretation

The equation contains two distinct terms.

8.1 Applied Torque

$$\boldsymbol{\tau}$$

represents external torques acting on the spacecraft, such as

- Reaction wheel torques
- Magnetorquer torques
- Gravity gradient torques
- Aerodynamic disturbance torques
- Solar radiation pressure torques

8.2 Gyroscopic Coupling

$$\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})$$

is the gyroscopic coupling term.

This term is responsible for several important rigid-body phenomena:

- Torque-free precession
- Nutation
- Intermediate-axis instability
- Tumbling motion of spacecraft

Without this term, the rotational dynamics would reduce to

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau}$$

which fails to capture the true behavior of rotating rigid bodies.

9 Implementation in Python

The simulator computes the angular momentum vector as

$$\mathbf{H} = \mathbf{J}\boldsymbol{\omega}.$$

Python implementation:

```
H = self.inertia_tensor @ omega
```

The gyroscopic coupling term is then computed as

$$\boldsymbol{\omega} \times \mathbf{H}.$$

Python implementation:

```
np.cross(omega, H)
```

Finally, the angular acceleration is computed using

$$\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{H}).$$

Python implementation:

```
omega_dot = self.inertia_tensor_inv @ (  
    torque - np.cross(omega, H)  
)
```

10 Conclusion

The rotational dynamics module of the DeiMOS ADCS simulator is based on Euler's rotational equations for rigid bodies. The resulting angular acceleration equation

$$\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1} [\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})]$$

forms the foundation for spacecraft attitude propagation and will later be integrated with quaternion kinematics, numerical integration schemes, attitude determination algorithms, and control systems.